

PROGRAMMAZIONE II

GESTIONE DEGLI ERRORI CON LE ECCEZIONI

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

SITUAZIONI ANOMALE

Anche il codice migliore può incorrere in scenari anomali che portano a errori

- ▶ Errori del sistema operativo
- ▶ Indisponibilità di risorse
- ▶ Errori di input dell'utente
- ▶ Comportamenti imprevisti

Tali errori dovrebbero essere individuati e gestiti correttamente

- ▶ Interrompere l'esecuzione
- ▶ Chiamare procedure di gestione degli errori
- ▶ ...

GESTIONE DEGLI ERRORI

```
float division(int num, int den) {           1
    float res;                               2

    if (num != 0)                            3
        res = num / den;                     4
    else                                     5
        res = Float.MAX_VALUE; // valore speciale restituito 6
    return res;                              7
}                                             8
                                           9
                                           10
```

Questa soluzione rende il codice non riutilizzabile

- Il chiamante potrebbe non accorgersi della situazione anomala

GESTIONE DEGLI ERRORI (CONT.)

È possibile restituire un valore speciale al chiamante

- ▶ Gli sviluppatori devono ricordare i codici di errore
- ▶ A volte il valore di errore è in realtà un risultato legittimo
- ▶ Non è adatto per i costruttori

Le cose sono ancora più complesse con chiamate annidate

- ▶ La posizione dell'errore è difficile da tracciare
- ▶ O quando il chiamato non è in grado di gestire l'errore

UN ONERE PER IL PROGRAMMATORE

Senza una funzionalità specifica del linguaggio, la gestione degli errori spetta al programmatore

- Il codice risulterà più disordinato

```
if (someFunc() == ERROR) { // il chiamato ha rilevato l'errore      1
    // il chiamante gestisce l'errore                                2
} else {                                                            3
    // esecuzione normale                                           4
}                                                                    5
```

Solo il **chiamante diretto** può intercettare l'errore (nessuna delega verso l'alto)

- A meno di non aggiungere ulteriore codice boilerplate

ECCEZIONI

Le situazioni anomale che si verificano durante l'esecuzione sono chiamate **eccezioni**

- ▶ Se non gestite correttamente, le eccezioni possono causare un crash
- ▶ Quando accade qualcosa di imprevisto, viene eseguito del **codice di ripristino**

Java fornisce blocchi **try** .. **catch** per gestire programmaticamente le eccezioni

- ▶ Insieme a specifiche classi `Exception`

GESTIRE LE ECCEZIONI

Gestione delle eccezioni in Java

- Try** Blocco di codice in cui possono verificarsi situazioni anomale
- Throw** L'eccezione (o eccezioni) sollevata quando si verificano situazioni anomale
- Catch** Blocco di codice che gestisce una situazione anomala

Flusso di lavoro

- ▶ Il programma tenta di eseguire del codice (**try**)
- ▶ Se accade qualcosa di anomalo, il programma solleva un'eccezione (**throw**)
- ▶ L'eccezione può essere catturata e viene eseguito del codice di ripristino (**catch**)

GESTIRE LE ECCEZIONI (CONT.)

```
int readFile (String fileName) {  
    // apre il file  
    if (operationError)  
        return -1;  
    // calcola dimensione file  
    if (operationError)  
        return -2;  
    // alloca memoria per file  
    if (operationError)  
        return -3;  
    // legge file in memoria  
    if (operationError)  
        return -4;  
    // chiude il file  
    if (operationError)  
        return -5;  
    return 0; // tutto è andato a buon fine  
}
```

```
try {  
    // apre il file  
    // calcola dimensione file  
    // alloca memoria per file  
    // legge file in memoria  
    // chiude il file  
} catch (fileOpenError) {  
    // gestisce errore apertura file  
} catch (fileSizeError) {  
    // gestisce errore dimensione file  
} catch (allocationError) {  
    // gestisce errore allocazione  
} catch (fileReadError) {  
    // gestisce errore lettura file  
} catch (fileCloseError) {  
    // gestisce chiusura file  
}  
// tutto è andato a buon fine
```

Codice della logica di business disaccoppiato dal codice di gestione errori

SOLLEVARE ECCEZIONI

Le eccezioni possono essere lanciate da librerie di sistema o personalizzate

- ▶ L'eccezione `FileNotFoundException` lanciata dalla classe `java.io.FileInputStream`
- ▶ Il codice utente deve gestirle

Le eccezioni possono essere lanciate dal codice utente

- ▶ Eccezioni già definite (es. `FileNotFoundException`)
- ▶ Eccezioni definite dall'utente, che estendono la classe `Exception`

GENERAZIONE DI ECCEZIONI

1. Dichiarare una classe di eccezione
2. Contrassegnare il metodo che solleva l'eccezione
3. Istanziare l'eccezione
4. Lanciare (throw) l'eccezione

```
public class EmptyStackException extends Exception { } // 1.
class Stack {
    public Object pop() throws EmptyStackException { // 2.
        if (stackSize == 0) {
            Exception e = new EmptyStackException(); // 3.
            throw e; // 4.
        }
        ...
    }
}
```

GENERAZIONE DI ECCEZIONI (CONT.)

I metodi che lanciano eccezioni devono dichiararle dopo la parola chiave **throws**

```
public Object pop () throws EmptyStackException, IOException { ... }
```

Il metodo deve dichiarare

- ▶ Le eccezioni lanciate direttamente dal metodo
- ▶ Le eccezioni lanciate dai metodi chiamati e **non catturate** dal metodo stesso

L'esecuzione di un metodo viene interrotta quando viene eseguito **throw**

INTERCETTARE LE ECCEZIONI

Le eccezioni vengono catturate quando si trovano nell'ambito di un blocco **catch**

```
try {
    stack.pop();
    // codice da eseguire se tutto va a buon fine
} catch (EmptyStackException e) {
    // gestisce l'errore
    System.out.println(e);
}
```

1
2
3
4
5
6
7

È possibile associare più blocchi **catch** a un blocco **try**

FLUSSO DI ESECUZIONE

Viene eseguito un solo blocco `catch`

```
File f new File("foo.txt");           1
try {                                  2
    f.open(); // solleva l'eccezione FileNotFoundException 3
    f.read();                             4
    f.close();                             5
} catch (FileNotFoundException fe) {    6
    System.out.println("File_error");    7
} catch (IOException ioe) {             8
    System.out.println("I/O_error");     9
// prosegue l'esecuzione                10
                                         11
```



FLUSSO DI ESECUZIONE

Viene eseguito un solo blocco `catch`

```
File f new File("foo.txt");           1
try {                                  2
    f.open();                          3
    f.read(); // solleva l'eccezione IOException 4
    f.close();                         5
} catch (FileError fe) {               6
    System.out.println("File_error");  7
} catch (IOException ioe) {            8
    System.out.println("I/O_error");   9
// prosegue l'esecuzione              10
//                                     11
```



ECCEZIONI PERSONALIZZATE

Nel costruttore possiamo personalizzare il comportamento dell'eccezione

```
1 public class EmptyStackException extends Exception {
2     public EmptyStackException(Stack stack) {
3         super("Pop Failure [stack]" + stack.toString() + " [empty]");
4     }
5 }
6
7 class Stack {
8     public Object pop() throws EmptyStackException {
9         if (stackSize == 0) {
10             throw new EmptyStackException(this);
11         }
12         ...
13     }
14 }
```


CATTURARE LE ECCEZIONI

Quando chiama codice che può sollevare un'eccezione, il chiamante può:

- ▶ Catturare direttamente l'eccezione
- ▶ Propagare l'eccezione al chiamante
- ▶ Catturare l'eccezione e rilanciarla

```
public class Dummy { 1
    public void foo() { 2
        try { // la riga seguente può sollevare l'eccezione FileNotFoundException 3
            FileReader f = new FileReader("file.txt"); 4
        } catch (FileNotFoundException fnf) { 5
            // cattura l'eccezione e fa qualcosa 6
        } 7
    } 8
} 9
```

CATTURARE LE ECCEZIONI (CONT.)

L'eccezione può essere ignorata dal chiamante e propagata verso l'alto

- Le eccezioni non catturate vengono propagate fino al `main()` e alla JVM

```

public class Dummy {
    public void foo() throws FileNotFoundException {
        // qui la gestione dell'eccezione è delegata al chiamante
        FileReader f = new FileReader("file.txt");
    }
}

public class MyClass {
    public static void main(String[] args) throws FileNotFoundException {
        Dummy d
        d.foo(); // questo può sollevare l'eccezione FileNotFoundException
    }
}

```

CATTURARE LE ECCEZIONI (CONT.)

Possiamo comunque propagare verso l'alto eccezioni catturate localmente

```

public class Dummy {
    public void foo() throws FileNotFoundException {
        try { // la riga seguente può sollevare l'eccezione FileNotFoundException
            FileReader f = new FileReader("file.txt");
        } catch (FileNotFoundException fnf) {
            // cattura l'eccezione e fa qualcosa
            throw fnf; // e la rilancia (propaga al chiamante)
        }
    }
}

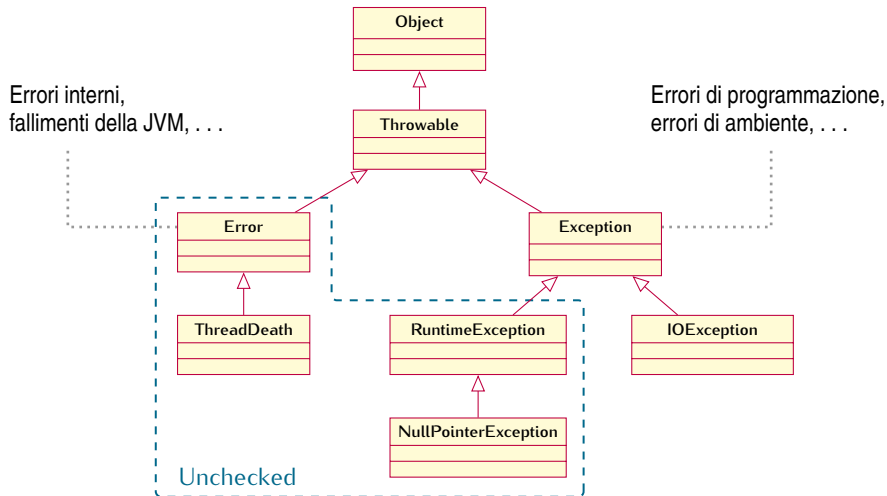
```

1
2
3
4
5
6
7
8
9
10

GERARCHIA DELLE ECCEZIONI



GERARCHIA DELLE CLASSI DI ECCEZIONE



ECCEZIONI CHECKED E UNCHECKED

Le eccezioni **checked** corrispondono a situazioni anomale che possiamo prevedere

- ▶ Esempio classico: `FileNotFoundException`
- ▶ Devono essere dichiarate, e quindi verificate dal compilatore
- ▶ Sono generate dal codice utente con `throw`
- ▶ Devono essere catturate dal chiamante

Le eccezioni **unchecked** corrispondono a situazioni anomale che non possiamo prevedere

- ▶ Esempio classico: `NullPointerException`
- ▶ Non devono essere dichiarate, e quindi non sono verificate dal compilatore
- ▶ Sono tipicamente generate dalla JVM
- ▶ Non devono essere catturate dal chiamante

ECCEZIONI CHECKED E UNCHECKED (CONT.)

Eccezioni checked

- ⊕ Chi usa un metodo capisce subito quali eccezioni possono essere sollevate
- ⊕ Il compilatore ci dice automaticamente se un'eccezione non è catturata
- ⊕ Il programma non terminerà mai a runtime, poiché le eccezioni devono essere gestite obbligatoriamente
- ⊖ Complicano la firma dei metodi

Eccezioni unchecked

- ⊖ Chi usa un metodo deve ispezionarne il corpo in dettaglio per cercare le eccezioni
- ⊖ Nessun supporto dal compilatore sulla gestione delle eccezioni
- ⊖ Il programma può terminare a runtime, poiché un'eccezione potrebbe non essere stata gestita
- ⊕ La firma del metodo rimane invariata

QUALE TIPO DI ECCEZIONE USARE

Le eccezioni checked si propagano come un virus

- ▶ Devono essere ricorsivamente dichiarate in tutti i contesti chiamanti
- ▶ Oppure catturate a un certo punto

Anche se più sicure, possono rendere il codice quasi illeggibile

- ▶ Il consiglio è di non definire eccezioni checked, a meno che non sia strettamente necessario

Per definire un'eccezione unchecked

- ▶ Basta estendere `RuntimeException` invece di `Exception`

TRUCCHI SPORCHI CON LE ECCEZIONI

Catturare un'eccezione checked e rilanciarla come eccezione unchecked

```
public class MyException extends Exceptions { } // checked 1
2
public void foo() throws MyException { 3
    throw new MyException(); 4
} 5
6
public void bar() throws MyException { // dichiarazione obbligatoria 7
    foo(); 8
    9
    10
    11
    12
} 13
```

TRUCCHI SPORCHI CON LE ECCEZIONI

Catturare un'eccezione checked e rilanciarla come eccezione unchecked

```
public class MyException extends Exceptions { } // checked 1
2
public void foo() throws MyException { 3
    throw new MyException(); 4
} 5
6
public void bar() { // nessuna dichiarazione richiesta 7
    try { 8
        foo(); 9
    } catch (MyException e) { 10
        throw new RuntimeException(e); // unchecked 11
    } 12
} 13
```

ECCEZIONI E CICLI

Per errori che riguardano singole iterazioni, il blocco `try` .. `catch` è annidato nel ciclo

- In caso di eccezione, si prosegue con la prossima iterazione

```
while (true) {                                1
    try {                                      2
        // codice che può sollevare un'eccezione  3
    } catch (Exception e) {                    4
        // gestisce l'errore locale              5
    }                                           6
}                                              7
```

ECCEZIONI E CICLI (CONT.)

Per errori che compromettono l'intero ciclo, il blocco `try` .. `catch` avvolge il ciclo

- In caso di eccezione, si esce dal ciclo

```
try {
    while (true) {
        // codice che può sollevare un'eccezione
    }
} catch (Exception e) {
    // gestisce l'errore globale
}
```

1
2
3
4
5
6
7

DOMANDE E RISPOSTE

